🚓

Modelado Matemático de Control Robótico: Un Enfoque en Espacio de Estadosb

Queremos controlar un carro de dos ruedas a través de las variables PWM, que regula la velocidad de los motores y su ángulo de giro y la medida de los tiempos de vuelo de las ondas generadas por un sensor ultrasónico.

Deberíamos usar la teoría de modelos. La realidad tiene dimensiones, pero nuestro modelo se basa en números puros. Por ejemplo, estamos usando PWM en lugar de corriente en los motores y esa corriente es la que determina la velocidad de giro. De la misma manera dado que la velocidad del sonido se puede considerar **constante** para nuestro entorno de operación y la **distancia es directamente proporcional al tiempo de vuelo** estrictamente no sería necesario multiplicar los tiempos para obtener distancias. En teoría, podemos trabajar directamente con los tiempos y evitar la conversión a milímetros.
La fórmula básica es:
`distancia = (velocidad_sonido * tiempo_vuelo) / 2`

Si `velocidad_sonido` es constante, entonces: distancia $\propto$ tiempo_vuelo

Por lo tanto, **el tiempo es una representación escalada de la distancia**. Cualquier algoritmo que use distancias puede usar tiempos aplicando el factor de escala correspondiente.

🔄 Implicaciones prácticas

Aspecto	Usando distancia (mm)	Usando tiempo (µs)
Unidades	mm	µs
Rango típico	20-4000 mm	~115-23500 µs
Precisión	~1 mm	~1 µs
Escala	1 mm ≈ 5.8 µs	1 µs ≈ 0.17 mm
Legibilidad	Más intuitivo	Menos intuitivo

Ventajas de usar tiempo:

- ✅ Una operación menos (menos cómputo, menor latencia)
- ✅ Evitas errores de redondeo en la conversión
- ✅ Los umbrales de peligro pueden definirse directamente en tiempo

🌡️ La velocidad del sonido NO es perfectamente constante

La velocidad del sonido varía con:

- **Temperatura**: ~0.6 m/s por cada °C (fórmula: $v = 331.3 + 0.606 * T$)
- **Humedad**: efecto pequeño pero existe
- **Presión atmosférica**: efecto mínimo

Para un robot que opera en interiores con temperatura estable (±5°C), el error es <3%. En exteriores con variaciones de 20°C, el error puede ser >12%.

La otra diferencia es que si obtenemos la velocidad como tiempo sobre tiempo sería adimensionada.

📐 Análisis dimensional

Magnitud	Usando distancia	Usando tiempo
Posición	L (mm)	T (µs)
Velocidad	L/T (mm/s)	T/T = **adimensional**
Aceleración	L/T^2	1/T

🧮 **Matemáticamente:**

```
distancia = k * tiempo  (donde k = velocidad_sonido/2)

velocidad_distancia = Δdistancia/Δt = k * (Δtiempo/Δt)
                    = k * velocidad_tiempo_adimensional

velocidad_tiempo_adimensional = Δtiempo/Δt  (sin unidades)
```

📊 Interpretación de la velocidad adimensional

- **speed_adim = 0.1**: El tiempo de vuelo aumenta un 10% del tiempo transcurrido → te alejas
- **speed_adim = -0.1**: El tiempo de vuelo disminuye un 10% del tiempo transcurrido → te acercas
- **speed_adim = 0**: Distancia constante

Deberíamos usar la teoría de modelos. La realidad tiene dimensiones, pero nuestro modelo se basa en números puros. Por ejemplo, estamos usando PWM en lugar de corriente en los motores y esa corriente es la que determina la velocidad de giro.

Lo que estamos construyendo es un **modelo matemático de control robótico** basado en la teoría de sistemas. Formalicémoslo con notación matemática.

📐 Definición del Sistema en Espacio de Estados

🎛️Variables de estado:

x_1 = tiempo_vuelo [μs] (posición en el espacio de señales)
x_2 = velocidad_adimensional (Δtiempo/Δt)

Vector de estado:
$x = [x_1, x_2]^T$

🎛️ Ecuaciones de Estado

[Evolución temporal] El tiempo en el intervalo de muestreo siguiente, será el tiempo actual mas el período de muestreo por la velocidad adimensional actual, a menos del ruido.

[Dinámica de velocidad] Mientras que la velocidad adimensional en el intervalo de muestreo siguiente, será una función del PWM aplicado actual y de la velocidad adimensional actual, a menos del ruido.

Lo que se puede expresar matemáticamente mediante las ecuaciones:

```
x₁(k+1) = x₁(k) + T·x₂(k) + w₁    [Evolución temporal]
x₂(k+1) = f(u(k), x₂(k)) + w₂     [Dinámica de velocidad]

donde:
T = período de muestreo [ms]
u = PWM aplicado [0-1023]
w₁, w₂ = ruido del proceso
```

📊 Modelo de Observación (Sonar)

```
y₁(k) = x₁(k) + v₁                    [Medición de tiempo]
y₂(k) = (x₁(k) - x₁(k-1))/T + v₂      [Velocidad derivada]

donde:
v₁, v₂ = ruido de medición
```

🎯 Ley de Control (PID como caso particular)

```
u(k) = Kp·e(k) + Ki·Σe(i) + Kd·(e(k)-e(k-1))
```

```
donde:
Kp es una constante de proporcionalidad
Ki es una constante de integración [suma de errores anteriores]
Kd es una constante de derivación [error actual - error anterior]
El error es la diferencia entre el valor deseado y el valor medido en el intervalos actual.
e(k) = r(k) - x₂(k)      [error de velocidad]
r(k) = referencia (target_avel del Master)
```

🔄 Sistema en Lazo Cerrado

```
┌─────────┐    ┌─────────┐    ┌─────────┐
│  r(k)   │──→│   PID   │──→│ Planta  │──→ x₂(k)
│ (Master)│    │         │    │ (Robot) │
└─────────┘    └─────────┘    └─────────┘
                    ▲                 │
                    │                 │
                    └─────────────────┘
                         y₂(k)
```

📈 Análisis Matemático

Función de transferencia del PID:

```
Gc(z) = Kp + Ki·(z/(z-1)) + Kd·((z-1)/z)
```

Modelo de la planta (simplificado):

```
Gp(z) = (b₁z + b₂)/(z² + a₁z + a₂)   [segundo orden]
```

Ecuación característica del lazo cerrado:

```
1 + Gc(z)·Gp(z) = 0
```

Para explicar estos tres pilares del control digital, lo mejor es verlos como una conversación entre el "cerebro" (el controlador), el "cuerpo" (la planta) y el "equilibrio" (la ecuación característica).

El controlador PID es el "cerebro" que toma decisiones basándose en tres tipos de información temporal:

1. El Controlador PID: Gc(z) (El Cerebro)

Esta es la regla que decide cuánto "esfuerzo" aplicar según el error detectado.

$$G_c(z) = K_p + K_i \frac{z}{z-1} + K_d \frac{z-1}{z}$$

- **Kp (Proporcional):** El "ahora". Reacciona al error actual de forma directa.
- **Ki z/(z-1) (Integral):** El "pasado". El término z/(z-1) actúa como un **acumulador**. En el mundo discreto, esto suma todos los errores anteriores para eliminar el error de estado estacionario (que el sistema llegue exactamente a donde quieres).
- **Kd (z-1)/z (Derivativo):** El "futuro". El término (z-1)/z es esencialmente una **resta entre el valor actual y el anterior**. Predice hacia dónde va el error para frenar el sistema y evitar que se pase de largo (sobreimpulso).

2. El Modelo de la Planta: Gp(z) (El Cuerpo)

Es la representación matemática de lo que quieres controlar (un motor, un brazo robótico, un sensor de temperatura).

$$G_p(z) = \frac{b_1 z + b_2}{z^2 + a_1 z + a_2}$$

- **Segundo Orden (z^2):** Significa que el sistema tiene "inercia" o dos elementos que almacenan energía. Por ejemplo, en un motor, sería la posición y la velocidad.
- **Los coeficientes a y b:** Definen la personalidad de la planta. Determinan qué tan rápido responde y si tiende a oscilar naturalmente.

- **El dominio z:** Recuerda que z^{-1} representa un **retraso de un paso de muestreo**. Una ecuación con z^2 necesita conocer los dos estados anteriores para calcular el siguiente.

3. Ecuación Característica: 1 + Gc(z)Gp(z) = 0 (El Destino)

Esta es la parte más crítica. Cuando cierras el lazo (retroalimentación), el comportamiento del sistema ya no depende solo de la planta o del control por separado, sino de su combinación.

- **¿Por qué igual a cero?** Los valores de z que resuelven esta ecuación son los **polos del sistema en lazo cerrado**.
- **La Regla de Oro de la Estabilidad:** Para que tu sistema no se vuelva loco y "explote" (inestabilidad), todas las soluciones (z) de esta ecuación deben vivir **dentro del círculo unitario** en el plano complejo.

Traducción rápida: Si al resolver 1 + Gc(z)Gp(z) = 0 obtienes un valor de $|z| > 1$, tu sistema es inestable. Si ajustas tus Kp, Ki, Kd correctamente, "empujas" esos valores dentro del círculo para que el control sea suave y preciso.

Resumen de la interacción

1. El **Error** entra al PID (Gc).
2. El PID envía una señal de **Control** a la planta (Gp).
3. La **Salida** de la planta se resta del objetivo para generar un nuevo error.
4. La **Ecuación Característica** define si este ciclo se estabiliza o se rompe.

- **Proporcional (Kp):** Reacciona al error actual - "¿qué tan lejos estoy ahora?"
- **Integral (Ki):** Acumula errores pasados - "¿he estado alejado mucho tiempo?"
- **Derivativo (Kd):** Anticipa la tendencia futura - "¿me estoy alejando o acercando?"

Matemáticamente, el PID en el dominio discreto (z) se expresa como:

🧮 Optimización y Sintonía

Podemos formular un problema de optimización:

```
min J = Σ [e(k)² + λ·Δu(k)²]
sujeto a:
0 ≤ u(k) ≤ 1023
|x₂(k)| ≤ x₂_max
```

🔬 Observador de Estado (Opcional)

Para estimar mejor la velocidad:

```
x̂(k+1) = A·x̂(k) + B·u(k) + L·(y(k) - C·x̂(k))

donde:
L = ganancia del observador (diseño por ubicación de polos)
```

🎓 Formalización del Control Distribuido

Master (planificación):

```
r(t) = g(θ, v_deseada)   [generación de referencias]
```

Slave (ejecución):

```
u(t) = h(e(t), ∫e dt, ė)  [control PID local]
x(t+1) = f(x(t), u(t))    [evolución del sistema]
y(t) = C·x(t)              [mediciones]
```

📝 Conclusión

Lo que hemos construido es un **sistema de control jerárquico**:

- **Nivel superior (Master)**: Genera referencias basadas en navegación
- **Nivel inferior (Slave)**: Control PID local para seguimiento preciso
- **Observaciones**: Tiempo de vuelo y velocidad adimensional
- **Actuación**: PWM en rango [0-1023]

Este es exactamente el enfoque de la **teoría moderna de control robótico**: separar la planificación (alto nivel) de la ejecución (bajo nivel con realimentación).

El modelo completo en espacio de señales

```
estado = [pwm_izq, pwm_der, ángulo_deseado]
observación = [tiempo_vuelo, tasa_aproximación]
objetivo = minimizar(tiempo_vuelo) o maximizar(tasa_aproximación > 0)
```

Control del ángulo de giro del robot

Si tenemos en cuenta que el ángulo de giro es función de la diferencia de los PWM izquierda y derecha, midiendo las distancias sucesivas en dos medidas veo si la distancia aumenta significa que debo girar en sentido contrario

Modelado matemático de la búsqueda de ángulo óptimo

Lo que has descubierto es un **observador de gradiente** para el sistema:

Estado: ángulo actual (desconocido)
Observación: distancia al obstáculo
Perturbación: pequeño cambio en ángulo
Objetivo: maximizar distancia (alejarse) o minimizar (acercarse perpendicularmente)

Este es un enfoque de **control sin modelo** que complementa perfectamente tu PID de velocidad.

Para incorporar la búsqueda del ángulo apropiado en tu modelo de espacio de estados, necesitamos **extender el vector de estado** para incluir la orientación y la dinámica de exploración. Aquí tienes una formulación matemática rigurosa:

📐 **Vector de estado extendido**

$$\mathbf{x} = \begin{bmatrix} x_1 & \text{(tiempo de vuelo)} \\ x_2 & \text{(velocidad adimensional)} \\ x_3 & \text{(ángulo actual)} \\ x_4 & \text{(gradiente estimado)} \end{bmatrix}^T$$

```
🔄 Ecuaciones de evolución
```

$$x_1(k+1) = x_1(k) + T \cdot x_2(k) + w_1 \tag{1}$$
$$x_2(k+1) = f(x_2(k), u_{pwm}(k)) + w_2 \tag{2}$$
$$x_3(k+1) = x_3(k) + u_{giro}(k) + w_3 \tag{3}$$
$$x_4(k+1) = \frac{x_1(k) - x_1(k-1)}{x_3(k) - x_3(k-1)} + w_4 \quad \text{(gradiente)} \tag{4}$$

🎯 **Ecuación de observación**

$$y(k) = x_1(k) + v_1 \quad \text{(medición del sonar)}$$

📊 Modelo de búsqueda de ángulo óptimo

El objetivo es encontrar el ángulo `x3*x3*` que **maximice el tiempo de vuelo** `x1 x 1` (alejarse del obstáculo) o lo minimice (acercarse perpendicularmente). Matemáticamente:

$$x_3^* = \arg\max_{x_3} \mathbb{E}[x_1 | x_3]$$

Podemos modelar la búsqueda como un proceso de **optimización basada en gradiente estocástico**:

1. **Estimación del gradiente local**:

```
\hat{\nabla}_{x_3}(k) = \frac{x_1(k) - x_1(k-1)}{x_3(k) - x_3(k-1)} \approx \frac{\Delta x_1}{\Delta x_3}
```

1. Esto es exactamente tu idea de medir cómo cambia la distancia al girar.
2. **Actualización del ángulo** (gradiente ascendente/descendente):

```
x_3(k+1) = x_3(k) + \alpha \cdot \hat{\nabla}_{x_3}(k) + \beta \cdot \eta(k)
```

donde:

- α = tasa de aprendizaje (qué tan rápido sigues el gradiente)
- β·η(k)= término de exploración (ruido para evitar óptimos locales)

1. **Para perpendicularidad** (distancia mínima): usar gradiente **descendente**
2. **Para alejarse** (distancia máxima): usar gradiente **ascendente**

🧮 Modelo completo en espacio de estados

```
\begin{cases}
\mathbf{x}(k+1) = \mathbf{x}(k) + \mathbf{u}(k) + \mathbf{w}(k) \\
y(k) = \mathbf{H}\mathbf{x}(k) + v(k)
\end{cases}
```

con:

$$u(k) = [T \cdot x2(k), f(x2, upwm), ugiro, 0]TH = [1, 0, 0, 0]H = [1, 0, 0, 0]\ (solomedimostiempodevuelo)$$

📈 Interpretación de las variables

Variable	Significado	Rango típico
x1$x1$	Distancia (tiempo de vuelo)	115-23500 µs
x2$x2$	Velocidad adimensional	-1.0 a 1.0
x3$x3$	Ángulo actual	0-359°
x4$x4$	Gradiente estimado	negativo = acercándose al girar

🎛️ Ley de control para búsqueda de ángulo

```
u_{giro}(k) = \gamma \cdot \text{signo}(x_4(k)) \cdot \min(|\text{ANGLE\_STEP}|, |x_4(k) \cdot K|)
```

donde:

- γγ = +1 para maximizar distancia, -1 para minimizar
- KK = ganancia de conversión gradiente → giro

🔄 Algoritmo completo

```
\begin{align}
x_4(k) &\approx \frac{x_1(k) - x_1(k-1)}{x_3(k) - x_3(k-1) + \epsilon} \\
x_3(k+1) &= x_3(k) + \gamma \cdot \alpha \cdot \tanh(x_4(k)) + \beta \cdot \mathcal{N}(0,1) \\
x_1(k+1) &= x_1(k) + T\cdot x_2(k) + \text{(efecto del giro)}
\end{align}
```

📊 Matriz de covarianza del sistema

Para un filtro de Kalman (opcional), definiríamos:

```
\mathbf{Q} = \begin{bmatrix}
\sigma_{w_1}^2 & 0 & 0 & 0 \\
0 & \sigma_{w_2}^2 & 0 & 0 \\
0 & 0 & \sigma_{w_3}^2 & 0 \\
0 & 0 & 0 & \sigma_{w_4}^2
\end{bmatrix}, \quad
\mathbf{R} = [\sigma_v^2]
```

🧪 Ejemplo numérico

Supongamos que estás girando hacia la izquierda:

$$x3(k) = 90^\circ x3(k) = 90^\circ,\ x3(k-1) = 85^\circ x3(k-1) = 85^\circ \ \rightarrow\ \Delta x3 = -5^\circ \Delta x3 = -5^\circ x1(k) = 1000\mu s x1(k)$$

📈 Interpretación de las variables

Variable	Significado	Rango típico
x1$x1$	Distancia (tiempo de vuelo)	115-23500 µs
x2$x2$	Velocidad adimensional	-1.0 a 1.0
x3$x3$	Ángulo actual	0-359°
x4$x4$	Gradiente estimado	negativo = acercándose al girar

🎛️ Ley de control para búsqueda de ángulo

$$u_{giro}(k) = \gamma \cdot \text{signo}(x_4(k)) \cdot \min(|\text{ANGLE_STEP}|, |x_4(k) \cdot K|)$$

🎛️

🎛️

🎯 Arquitectura

```
MASTER (ESP32 Master)                    SLAVE (ESP32 Slave)
│                                         │
│  Comando de alto nivel:                │
│  - angle: 90° (adelante)               │
│  - target_avel: 0.3 (velocidad deseada)│
│  ──────────────────────────────►       │
│                                         │
│                                         │ • Control PID local
│                                         │ • Lectura sonar → avel real
│                                         │ • Ajuste PWM en tiempo real
│                                         │ • No necesita preguntar al Master
│                                         │
│  ◄──────────────────────────────       │
│  (solo telemetría opcional)            │
```

📊 Flujo de datos completo

```
┌──────────────┐
│  MASTER      │
│ (decisiones  │
│  alto nivel) │
└──────┬───────┘
       │ SPI (50Hz)
       │ cmd.target_avel
       │ cmd.angle
       ▼
┌──────────────┐
│  SLAVE       │ ◄──────── Sonar (50Hz)
```

```
│ (control                │       avel_real
│  PID local)             │
└──────────┬──────────┘
           │ PWM (alta freq.)
           ▼
┌──────────┐
│ MOTORES  │
└────────
```

🎁 Ventajas de este diseño

1. **Bajo acoplamiento**: El Master no necesita saber cómo se controlan los motores
2. **Tiempo real**: El Slave reacciona en microsegundos, no espera al ciclo SPI
3. **Robustez**: Si el SPI falla, el Slave mantiene el último comando válido
4. **Simplicidad**: El Master solo manda "quiero ir a X velocidad en Y dirección"